



レシート管理システム

JavaScript / React / Next.js 初心者向け解説資料

JavaScript

TypeScript

React

Next.js 16

MySQL

Tailwind CSS

2026年5月

目次

章	内容
1	このアプリの概要と機能
2	使用技術の全体像
3	JavaScript と TypeScript
4	React — コンポーネントで画面を作る
5	useState — 画面の「状態」を管理する
6	useEffect — タイミングに応じた処理
7	Next.js — ファイルがそのままURLになる
8	Server Components と Client Components
9	API Routes — バックエンドを書く
10	データベース設計
11	データの流れ（全体フロー）
12	各画面の解説

この資料の読み方

ポイント

プログラミング未経験～半年程度の方を対象にしています。難しい部分は読み飛ばして構いません。図を中心に「どんな仕組みで動いているか」のイメージをつかむことを目標にしてください。

アプリの完成画面イメージ

レシート管理システム						レシート一覧	商品マスタ	+ 新規登録
ID	店舗名	購入日	税率	小計	合計			
1	スーパーABC	2026-05-01	10%	1,200	1,320 円	編集		
2	コンビニXYZ	2026-04-28	10%	850	935 円	編集		

▲ レシート一覧画面のイメージ

1. このアプリの概要と機能

何ができるアプリ？

お店で受け取ったレシートをデジタルで管理するWebアプリです。スマートフォンやパソコンのブラウザから操作でき、データはサーバーのデータベースに保存されます。

📄 レシート管理

- 一覧表示・新規登録・編集・削除
- 購入日から消費税率を自動計算
- 明細行ごとに税率設定が可能

📦 商品マスタ管理

- 商品コード・商品名・カテゴリを登録
- メーカー情報も管理
- カテゴリ：食料・お菓子・日用品など

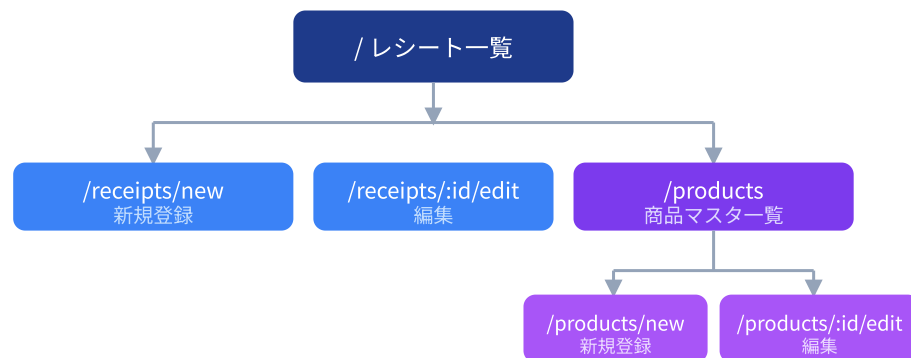
🔍 商品選択モータル

- 明細入力時にポップアップで商品検索
- カテゴリ絞り込み・テキスト検索
- 選択すると商品名が自動入力

💰 消費税の自動適用

- 購入日に応じた税率を自動取得
- 3%/5%/8%/10%に対応
- 明細ごとに個別税率を設定可能

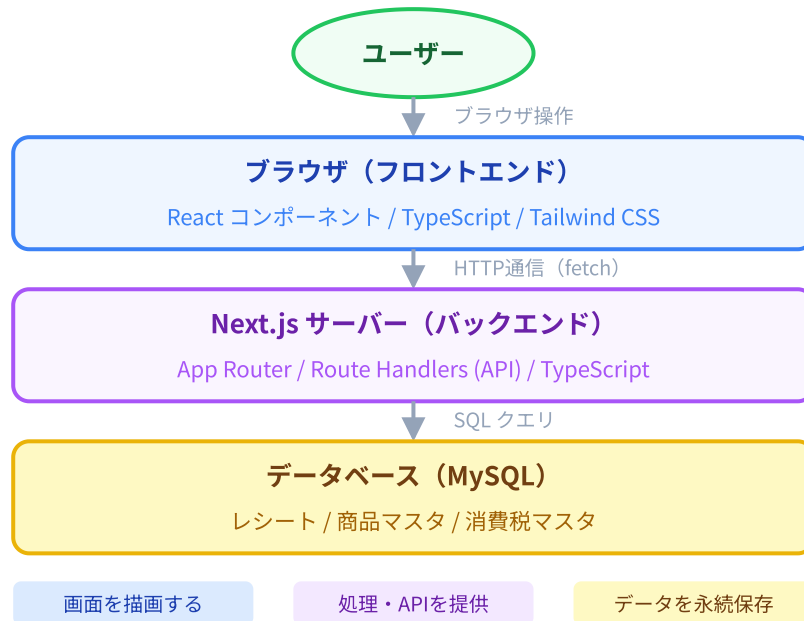
画面の一覧



▲ 画面（URL）の構成図

2. 使用技術の全体像

このアプリは複数の技術を組み合わせて作られています。それぞれが異なる役割を担っています。



▲ アーキテクチャ全体図 (3層構造)

技術スタック早見表

技術	役割	たとえば
JavaScript	Webブラウザで動くプログラミング言語	建物の「設計図の言語」
TypeScript	JavaScriptに型チェックを追加	設計図に「寸法の単位」を明記
React	UI部品（コンポーネント）を組み合わせてページを作る	LEGOブロックで家を建てる
Next.js	Reactをベースにしたフルスタックフレームワーク	建設現場の管理システム
MySQL	データを保存するデータベース	大きな電子台帳
Tailwind CSS	CSSをクラス名で書けるライブラリ	塗り絵の色鉛筆セット

3. JavaScript と TypeScript

JavaScript とは？

JavaScriptはWebブラウザの中で動くプログラミング言語です。ボタンをクリックしたときの動作や、サーバーとのデータのやり取りなどを記述します。

TypeScript とは？

TypeScript = JavaScript + 型

「型」とは、変数に入れられるデータの種類（文字列・数値・真偽値など）を宣言することです。型を明示することで**バグを事前に防ぎ**、コードを読みやすくします。このプロジェクトでは全ファイルをTypeScriptで書いています。

JavaScript（型なし）

```
// 何の型でも入れられる
let name = "山田太郎";
name = 123; // エラーにならない！

function add(a, b) {
  return a + b;
}
add("1", 2); // "12" になってしまう
```

TypeScript（型あり）

```
// 型を宣言する
let name: string = "山田太郎";
name = 123; // ← ここでエラーになる

function add(a: number, b: number): number {
  return a + b;
}
add("1", 2); // ← ここでエラーになる
```

このプロジェクトの型定義（src/lib/types.ts）

アプリ全体で使うデータの「形」を1ファイルに集約しています。

```
// 商品マスタの型
export type Product = {
  id?: number; // ? は「省略可能」を意味する
  product_code: string; // 商品コード（必須）
  product_name: string; // 商品名（必須）
  category_id?: number | null; // カテゴリID（省略可）
  manufacturer?: string; // メーカー（省略可）
};

// レシート明細の型
export type ReceiptItem = {
  product_code?: string; // 商品コード（省略可）
  item_name: string; // 商品名（必須）
  quantity: number; // 数量
  unit_price: number; // 単価
  amount: number; // 金額（数量×単価）
  item_tax_rate: number; // 税率（例：10）
}
```

```
item_tax_amount: number; // 税額  
};
```

✅ まとめ

TypeScript の「型」は**設計書**のようなものです。「この変数には数値しか入らない」と宣言しておくことで、間違った値を入れようとしたとき、コードを書いている段階でエラーとして気づけます。

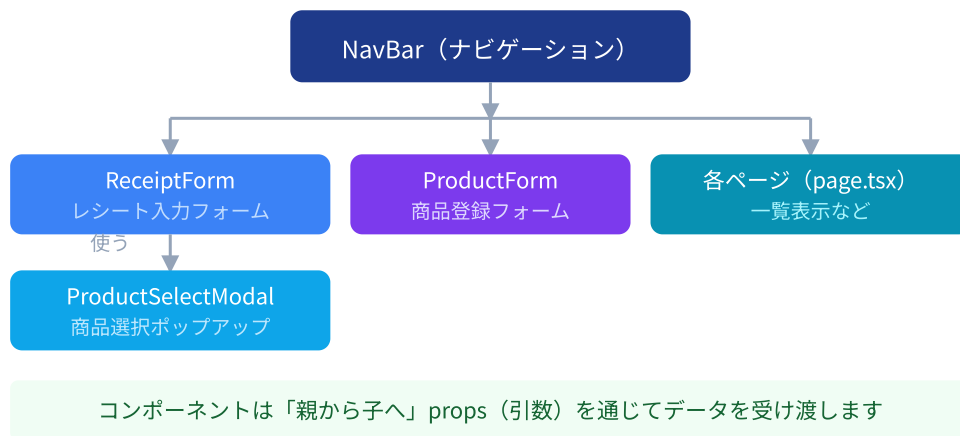
4. React — コンポーネントで画面を作る

コンポーネントとは？

💡 コンポーネント = 画面の「部品」

Reactでは、画面をバラバラの部品（コンポーネント）に分割して作ります。ボタン・フォーム・テーブルそれぞれが独立した部品です。LEGOブロックのように組み合わせて1つの画面を作ります。

このアプリのコンポーネント構成



▲ コンポーネント構成図

コンポーネントの書き方（例）

```
// src/components/NavBar.tsx
import Link from 'next/link';

// コンポーネントは「関数」として定義する
export default function NavBar() {
  return (
    {/* JSX: JavaScript の中に HTML のような記法を書ける */}
    <header className="bg-blue-600 text-white">
      <h1>レシート管理システム</h1>
      <nav>
        <Link href="/">レシート一覧</Link>
        <Link href="/products">商品マスタ</Link>
      </nav>
    </header>
  );
}
```

⚠️ JSX について

Reactコンポーネントの中では `<div>` のようなHTMLに見える記法が使えます。これを **JSX** とい
い、実際はJavaScriptに変換されます。HTMLとの違いは `class` の代わりに `className` を使う点
などです。

5. useState — 画面の「状態」を管理する

「状態 (State)」とは？

Reactでは、画面に表示される変化するデータを「状態 (State)」と呼びます。テキストボックスの入力値、チェックボックスのオン/オフ、APIから取得したデータなどが該当します。

重要：状態が変わると画面が自動で再描画される

`useState` で管理している値が変わると、Reactは自動的に画面（コンポーネント）を再描画します。開発者は「状態を更新する」だけで、画面の更新は自動です。

useState の仕組み



このアプリでの使用例

```
// src/components/ReceiptForm.tsx (抜粋)

// useState で状態を宣言する
// [現在の値, 値を更新する関数] の形で受け取る
const [storeName, setStoreName] = useState(''); // 店舗名
const [purchaseDate, setPurchaseDate] = useState(''); // 購入日
const [items, setItems] = useState([]); // 明細リスト
const [submitting, setSubmitting] = useState(false); // 送信中フラグ

// ← 入力値が変わったら setStoreName() で状態を更新
// → React が自動で画面を再描画してくれる

/>
```

✔ ポイント

- 変数を直接書き換えない（`storeName = "ABC"` はNG）
- 必ずセッター関数を使う（`setStoreName("ABC")` がOK）

- セッター関数を呼ぶと、Reactが自動的に画面を最新の状態に更新してくれる

6. useEffect — タイミングに応じた処理

useEffect とは？

コンポーネントが画面に表示されたとき、または特定の値が変化したときに、**自動的に実行したい処理**を書くための仕組みです。「副作用（Side Effect）」と呼ばれます。

よく使う場面

- 画面表示時にAPIでデータを取得
- 入力値が変わったらAPIを叩く
- タイマーのセット・解除

⚠ 注意点

- 第2引数の配列が依存関係
- `[]` のとき → 初回のみ実行
- `[value]` → value変化時に実行

購入日変更 → 税率取得の例



```
// 購入日が変わったら自動的に税率を取得する
useEffect(() => {
  if (purchaseDate) fetchTaxRate(purchaseDate);
}, [purchaseDate]); // ← purchaseDate が変化するときだけ実行

// デバウンス（タイピング中の過剰な通信を防ぐ）
useEffect(() => {
  const timer = setTimeout(fetchProducts, 300); // 0.3秒待つ
  return () => clearTimeout(timer); // クリーンアップ
}, [search, categoryId]);
```

📦 デバウンスとは？

商品検索で1文字入力するたびにAPIを叩くと、サーバーへの負荷が高くなります。「0.3秒間入力が止まったらAPIを叩く」という仕組みがデバウンスです。`setTimeout` のクリーンアップで実現しています。

7. Next.js — ファイルがそのままURLになる

App Router のファイルベースルーティング

Next.js では `src/app/` フォルダのディレクトリ構造がそのままURLになります。ファイルを置くだけでページが作れる仕組みです。



▲ ファイル構成とURLの対応

[id] — 動的ルーティング

フォルダ名を `[id]` のように角括弧で囲むと、URLの一部を変数として受け取れます。

```
// src/app/receipts/[id]/edit/page.tsx
type Props = { params: Promise<{ id: string }> };

export default async function EditReceiptPage({ params }: Props) {
  const { id } = await params; // URLの [id] 部分を取得
  // /receipts/3/edit にアクセスすると id = "3" になる
  const receipt = await getReceipt(id);
  // ...
}
```

⚠ Next.js 15以降の変更点

Next.js 15 から `params` は **Promise** として渡ってきます。 `await params` で中身を取り出す必要があります（古い書き方では `params.id` と直接アクセスできましたが、現在はエラーになります）。

8. Server Components と Client Components

2種類のコンポーネント

Next.js の App Router では、コンポーネントをサーバーで動かすか、ブラウザで動かすかを選べます。

Server Components (デフォルト)

- サーバー側で実行される
- DBへの直接アクセスが可能
- `useState` / イベントが使えない
- ファイルの先頭に何も書かない

例：一覧ページ（DBからデータ取得して表示）

Client Components

- ブラウザ側で実行される
- `useState` / `useEffect` が使える
- ボタンクリックなどのイベント処理
- ファイルの先頭に `'use client'` と書く

例：フォーム（ユーザー入力を受け付ける）

Next.js サーバー

products/page.tsx
Server Component

- ✓ DBに直接アクセス
- ✓ 高速（JS不要）
- ✗ インタラクション不可

ブラウザ

ReceiptForm.tsx
Client Component ('use client')

- ✓ `useState` / `useEffect`
- ✓ ボタンクリックなど
- ✗ DBに直接アクセス不可

このアプリでの使い分け

ファイル	種別	理由
app/products/page.tsx	Server	DBから商品一覧を取得して表示するだけ
components/ProductForm.tsx	Client	入力フォーム・バリデーション・状態管理が必要
components/ProductSelectModal.tsx	Client	検索・フィルター・クリックイベントが必要
components/NavBar.tsx	Server	Linkタグを並べるだけ・状態不要

9. API Routes — バックエンドを書く

Route Handlers とは？

`src/app/api/` フォルダに `route.ts` を置くと、そのファイルがAPIエンドポイントになります。ブラウザからの HTTP リクエストを受け取り、DBとやり取りしてデータを返します。



Route Handler の書き方

```
// src/app/api/products/route.ts
import { NextResponse } from 'next/server';
import pool from '@/lib/db'; // DBコネクション

// GET /api/products → 商品一覧を返す
export async function GET() {
  const [rows] = await pool.query(`SELECT * FROM products`);
  return NextResponse.json(rows); // JSON形式で返す
}

// POST /api/products → 商品を新規登録する
export async function POST(request: Request) {
  const body = await request.json(); // リクエストのデータを取得
  await pool.query(
    `INSERT INTO products (product_code, product_name) VALUES (?, ?)`,
    [body.product_code, body.product_name]
  );
  return NextResponse.json({ ok: true }, { status: 201 });
}
```

✓ RESTful API の HTTP メソッド

GET データを取得する

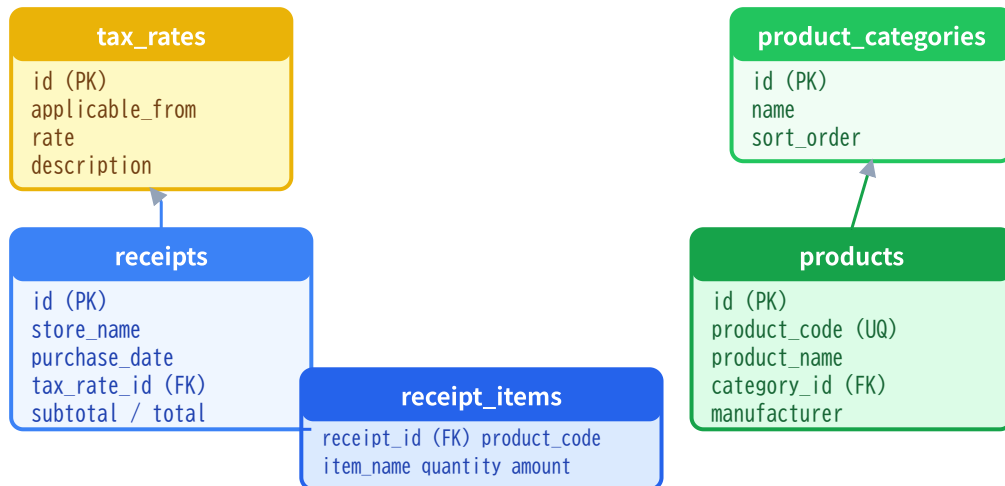
POST データを新規作成する

PUT データを更新する

DELETE データを削除する

10. データベース設計

テーブル構成 (ER図)



※ receipt_items.product_code は外部キーなし (疎結合)

💡 疎結合の設計意図

receipt_items の **product_code** は、商品マスタとの外部キー制約を設けていません。商品を削除した後でも、過去のレシート記録は影響を受けないためです。商品マスタはあくまで「入力の補助ツール」として位置づけています。

消費税マスタの使い方

日本の消費税率の歴史を **tax_rates** テーブルで管理します。購入日以前で最も新しい税率レコードを取得する仕組みです。

```
-- 2023-01-01 の購入なら 10% が返る
SELECT * FROM tax_rates
WHERE applicable_from <= '2023-01-01'
ORDER BY applicable_from DESC
LIMIT 1;
```


11. データの流れ（全体フロー）

商品をモーダルから選択してレシートを登録するまで



💡 トランザクションとは？

レシートヘッダと明細の両方をDBに保存するとき、途中でエラーが起きると「ヘッダだけ保存された中途半端な状態」になってしまいます。トランザクションを使うことで、**全部成功か、全部失敗か**（アトミック）を保証できます。

12. 各画面の解説

① 商品マスタ登録画面（/products/new）

レシート管理システム

商品マスタ 新規登録

商品コード *

4901234567890

商品名 *

緑茶 500ml

カテゴリ

嗜好品 ▼

メーカー

サントリー

登録する

キャンセル

② レシート明細行（商品コード入力 + 選択ボタン）

商品コード / 商品名	数量	単価	金額	税率
4901234567890 緑茶 500ml 選択	1	198	198	10%

↑ モーダルから選択するか、コードを直接入力してEnter

③ 商品選択モーダル（ポップアップ）

商品選択

商品コード・商品名・メーカーで検索

全カテゴリ ▼

商品コード	商品名	カテゴリ	メーカー
4901234567890	緑茶 500ml	嗜好品	サントリー
4902430.....	コーヒー牛乳	嗜好品	雪印

✓ このアプリで学べること

- ✓ TypeScript で型安全なコードを書く
- ✓ useState / useEffect の活用
- ✓ API Routes でバックエンドを実装

- ✓ React のコンポーネント設計
- ✓ Next.js App Router のルーティング
- ✓ MySQL との接続・トランザクション